DAY 14 / 15 · PHASE: ADVANCED

Zero-knowledge proofs & smart-contract wallets (ERC-4337)

Study guide · 28 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-14

1. Learning objectives

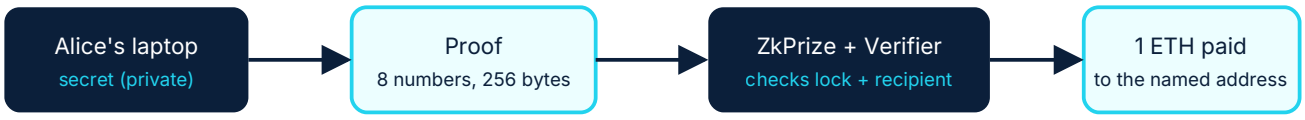
- Explain a zero-knowledge proof in plain words and name its parts: circuit, witness, proof, verifier, public signals.
- Build a Groth16 proof with circom + snarkjs and verify it on-chain to pay out a prize.
- Show why a proof must name its recipient (front-running) and why a weak secret is not protected.
- Explain how ERC-4337 works (UserOperation, bundler, EntryPoint, wallet) and build a wallet with a daily limit and guardian recovery.
- Audit like an attacker: separation of duties, gas as a hole in a spending limit, boundary values.

2. The concept, step by step

Part 1: Zero-knowledge in plain words

A zero-knowledge proof convinces someone that a statement is true **without showing why**. Comparison: you prove you know the Wi-Fi password by connecting; your friend learns that you know it, not the password.

Today's statement: *"I know a secret* `s` *with Poseidon(s) = this public lock, and the prize goes to **this address**."*



Word	Meaning	Comparison
Circuit	The statement as maths rules (constraints). Ours: 416 constraints.	A form whose boxes must add up
Witness	All numbers that fill the form, including the secret. Stays at home.	Your filled-in form
Proof	256 bytes showing the form was filled correctly	A notary's stamp
Verifier	Contract generated by snarkjs that checks the stamp	The bank checking the stamp
Public signals	[lock, recipient]: what everybody sees	The outside of the envelope

Part 2: Account abstraction (ERC-4337)

A normal account (EOA) *is* its key: lose it, money gone; steal it, everything gone at once. A smart wallet is a **contract** with its own rules: cash in your pocket vs a bank account with an ATM limit and a recovery desk. ERC-4337 adds this without changing Ethereum itself.

Piece	Job
UserOperation	A signed "please do this" from the owner (not a transaction)
Bundler	Collects UserOperations, sends one `handleOps` transaction, is refunded for gas
EntryPoint	One official contract: asks the wallet to validate, runs the call, refunds the bundler
Smart wallet	`validateUserOp` (is this signature mine? pay gas) and `execute` (do it, within my rules)

Flow: owner signs → bundler → `EntryPoint.handleOps` → wallet `validateUserOp` → `execute` → daily limit checked → ETH sent.

3. Key code, annotated

```
template ProveSecret() {
    signal input secret;      // PRIVATE
    signal input hash;        // public: the lock
    signal input recipient;   // public: binds the proof to one address
    component h = Poseidon(1);
    h.inputs[0] <== secret;
    hash === h.out;           // rule: Poseidon(secret) must equal the lock
    signal recipientSquare;
    recipientSquare <== recipient * recipient; // keep recipient in a real constraint
}
```

```
uint256[2] memory pub = [secretHash, uint256(uint160(address(recipient)))];
if (!verifier.verifyProof(a, b, c, pub)) revert InvalidProof();
claimed = true;                 // effects BEFORE the external call
(bool ok, ) = recipient.call{value: amount}("");
```

```
function execute(address dest, uint256 value, bytes calldata data) external onlyEntryPointOrOwner
{
    if (dest == address(_entryPoint)) revert ForbiddenTarget(dest); // QA M1
    _spend(value);                                                  // daily limit
    (bool ok, bytes memory ret) = dest.call{value: value}(data);
    if (!ok) revert CallFailed(ret);
}
```

4. Results from the run

What	Result
Proof size / build time	256 bytes (8 numbers) / 642 ms, whatever the secret
Verify + pay out on-chain	258,782 gas
Mallory copies the proof, names herself	rejected: `InvalidProof`
Naive prize (no recipient in proof)	copier wins the 1 ETH without knowing the secret
4-digit PIN as the secret	cracked in 4,822 guesses / 257 ms
Wallet: signed by someone else / same op twice	AA24 signature error / AA25 invalid nonce
Wallet: 0.6 ETH more with 0.5 left	inner call fails (`DailyLimitExceeded`), gas still paid
Guardian recovery after 2 days	new owner works, old key gets AA24
Tests	52 passing (43 contract + 9 exercise)

5. QA & security findings

ID	Sev.	Finding	Fix
M1 (morning)	Medium	Daily limit bypassed through gas: the prefund is paid in `validateUserOp` before `execute`; then `withdrawTo` via `execute` with value 0 pulled ~4.5 ETH out	Cap gas per op at 0.01 ETH; forbid calls to the EntryPoint
L1 (evening)	Low	Owner and guardian could be the same address, so one stolen key could also raise the limit	`OwnerIsGuardian`, `AlreadyOwner` checks
L2 (evening)	Low	A proof naming the prize contract as recipient locked the ETH forever	`RecipientIsPrize` check
K7 (known)	Design limit	"Per day" = per UTC calendar day: 2 ETH left in 35 s around midnight	Not fixed; rolling-window model in exercise 3

Tools: ESLint clean · Solhint clean · `npm audit --omit=dev` 0 vulnerabilities · Slither 9 results, all Info.

6. Common mistakes & interview questions

- Leaving the recipient / `msg.sender` out of the public inputs, so proofs can be stolen from the mempool.
- Using a guessable secret: the lock is public, so anyone can hash guesses.
- One-person trusted setup used with real money; unconstrained public inputs in circuits.

- Forgetting that wallet validation pays gas before any `execute` rule runs.
- Giving one address two roles that were meant to check each other.

1. What does a ZK proof reveal? Only that the statement is true, plus the public signals. Not the secret.

2. Why must the recipient be a public input? A proof is plain data in the mempool. Without the recipient bound into it, anyone can copy it, pay a higher fee and claim the prize.

3. What is a trusted setup? Groth16 needs per-circuit keys made from random "toxic waste"; whoever keeps it can forge proofs. Real projects run a multi-party ceremony: one honest participant is enough.

4. What are the parts of ERC-4337? UserOperation (signed intent), bundler (submits them), EntryPoint (validates and runs them, refunds gas), smart wallet (rules), optional paymaster (pays gas).

5. Why return SIG_VALIDATION_FAILED instead of reverting on a bad signature? Bundlers simulate operations; a revert breaks gas estimation. The EntryPoint then reports AA24 itself.

7. Exercises (evening)

1. **Easy:** brute-force an n-digit PIN from its public lock and estimate the time for 248 random bits (~5e62 years).
2. **Medium:** a sliding-window function that finds the most sent in any window; show the midnight loophole (2 ETH in 60 s).
3. **Challenge:** a `RollingLimit` model: never more than the limit in any 24 h, proven with a property test.

Cheat sheet

- ZK = prove it without showing it. Proof = 256 bytes, verify ≈ 259k gas.
- Public inputs must name who is paid. Secrets need ≥ 128 random bits.
- Groth16 = trusted setup per circuit. Poseidon = hash built for circuits.
- ERC-4337: UserOp → bundler → EntryPoint → `validateUserOp` → `execute`.
- Errors: AA24 = bad signature, AA25 = bad nonce. Validation pays gas first.
- Roles that check each other must be different addresses. Test boundaries (midnight!).

8. References

1. circom documentation: https://docs.circom.io/
2. snarkjs (iden3): https://github.com/iden3/snarkjs
3. ethereum.org, Zero-knowledge proofs: https://ethereum.org/en/zero-knowledge-proofs/
4. Groth, "On the Size of Pairing-based Non-interactive Arguments" (2016): https://eprint.iacr.org/2016/260
5. EIP-4337, Account Abstraction: https://eips.ethereum.org/EIPS/eip-4337
6. ERC-7562, Validation scope rules: https://eips.ethereum.org/EIPS/eip-7562
7. eth-infinitism account-abstraction (EntryPoint v0.7): https://github.com/eth-infinitism/account-abstraction
8. OpenZeppelin Contracts, ECDSA: https://docs.openzeppelin.com/contracts/5.x/api/utils#ECDSA
9. Solidity documentation: https://docs.soliditylang.org/
10. Trail of Bits, zero-knowledge security: https://github.com/trailofbits/publications